

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

дисциплина: Архитектура компьютера —

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Системы контроля версий. Общие понятия

2.2 Система контроля версий Git

2.3 Основные команды git

Выполнения лабораторной работы

3.1 Выполнения лабораторной работы

3.2 Базовая настройка git

3.3 Создание SSH-ключа

3.4 Создание рабочего пространства и репозитория курса

3.5 Настройка каталога курса

Выполнение самостоятельной работы

Выводы

Список литературы

Цель работы

Целью работы является изучение идеологии и применения средств контроля версий, приобретение практических навыков по работе с системой контроля версий git.

Теоретическое введение

2.1 Системы контроля версий. Общие понятия

Системы контроля версий (СКВ) — это программные инструменты, предназначенные для записи изменений в файлы (чаще всего исходный код) с возможностью последующего возврата к любой сохраненной версии. Их основная цель — управление эволюцией проекта, позволяющее нескольким разработчикам работать параллельно, не мешая друг другу, отслеживать всю историю модификаций, а также выявлять и разрешать конфликты при слиянии изменений. Ключевые концепции включают репозиторий (хранилище всех версий файлов и метаданных), коммит (фиксация изменений с уникальным идентификатором), ветвление (создание независимых линий разработки) и слияние (объединение изменений из разных веток). Современные СКВ делятся на централизованные (например, Subversion), где существует единый сервер-репозиторий, и распределенные (такие как Git и Mercurial), где каждый участник имеет полную локальную копию репозитория, что обеспечивает большую гибкость и отказоустойчивость.

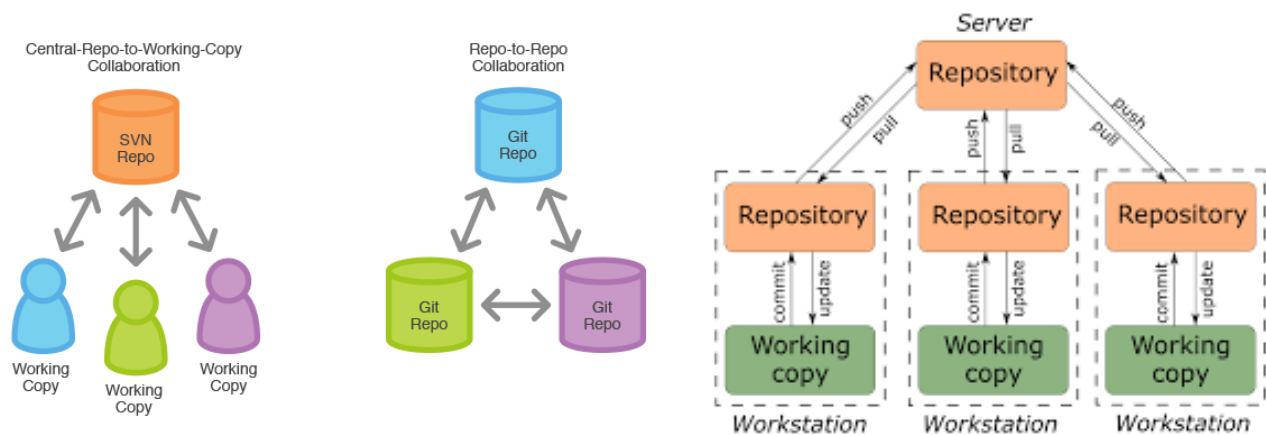
Использование СКВ стало неотъемлемым стандартом в разработке программного обеспечения, управлении документацией и даже в научных исследованиях. Эти системы не только предоставляют механизм «отката» к рабочей версии, но и формируют структурированную историю проекта, где каждый коммит сопровождается комментарием, что значительно упрощает анализ причин изменений и поиск ошибок. Благодаря ветвлению команды могут экспериментировать с новыми функциями в изолированных средах, а затем интегрировать их в основную линию кода. Таким образом, СКВ являются фундаментальным элементом DevOps-практик, обеспечивая непрерывную интеграцию, коллаборацию и надежное управление любыми цифровыми активами.

2.2 Система контроля версий Git

Git — это распределенная система контроля версий, созданная Линусом Торвальдсом в 2005 году для управления разработкой ядра Linux. В отличие от централизованных систем, Git предоставляет каждому разработчику полноценную локальную копию всего репозитория, включая всю его историю. Это позволяет работать автономно, выполнять коммиты, создавать ветки и просматривать историю без необходимости сетевого подключения к серверу. Ключевая архитектурная особенность Git — хранение данных как последовательности

снимков (snapshots) файловой системы, а не как списка изменений (дельта). Каждый коммит содержит ссылку на полный снимок всех файлов на момент его создания, что делает операции (например, сравнение версий или переключение между ними) чрезвычайно быстрыми. Основные команды и концепции включают `git init` (инициализация репозитория), `git add` (подготовка изменений), `git commit` (фиксация), `git branch` (работа с ветками), `git merge` и `git rebase` (слияние истории), а также `git clone` и `git push/git pull` для синхронизации с удаленными репозиториями.

Одной из самых мощных и отличительных возможностей Git является его модель ветвления. Создание веток в Git — это быстрая и легкая операция, поощряющая нелинейные рабочие процессы, такие как Feature Branch Workflow или Git Flow. Это позволяет командам вести параллельную разработку функций, исправлений и экспериментов, а затем интегрировать их через механизмы слияния или перебазирования. Git также гарантирует целостность данных: каждый коммит и объект в Git идентифицируется криптографическим хешем SHA-1, что делает историю неизменяемой и защищенной от непреднамеренных искажений. Благодаря своей скорости, гибкости и децентрализованной природе, Git стал де-факто мировым стандартом для контроля версий, являясь основой для платформ совместной работы, таких как GitHub, GitLab и Bitbucket, которые добавляют к нему функции pull request, code review и CI/CD.



2.2 Основные команды git

В данной таблице будут представлены все основные команды Git

КАТЕГОРИЯ	КОМАНДА	КРАТКОЕ ОПИСАНИЕ	ПРИМЕР
ИНИЦИАЛИЗАЦИЯ И НАСТРОЙКА	git init	Инициализирует новый репозиторий Git в текущем каталоге.	git init
	git config	Настраивает параметры Git (пользователь, редактор и т.д.).	git config --global user.name "Ваше Имя"
	git status	Показывает состояние рабочего каталога и индекса (staging area).	git status
	git log	Показывает историю коммитов репозитория.	git log --oneline --graph
СОСТОЯНИЕ И СПРАВКА	git --help	Показывает общую справку или справку по конкретной команде.	git commit --help
	git add	Добавляет изменения из рабочего каталога в индекс (область подготовленных файлов).	Git add мой_файл.py git add .
	git commit	Фиксирует (сохраняет) подготовленные изменения в локальном репозитории.	git commit -m "Описательное сообщение"
	git restore	Отменяет изменения в рабочем каталоге (не в индексе).	git restore измененный_файл.txt
ОТСЛЕЖИВАНИЕ ФАЙЛОВ	git rm	Удаляет файл	git rm файл_на_удаление.txt

**ВЕТКИ
(BRANCHES)**

git branch	из рабочего каталога и индекса. Показывает список, создаёт или удаляет ветки	git branch git branch новая-ветка
git checkout	Позволяет переключить ся на другую ветку или восстановить файлы.	git checkout другая-ветка
git switch	Новая команда для более интуитивног о переключени я между ветками.	git switch main
git merge	Вливает изменения из указанной ветки в текущую ветку.	git merge feature-1

**УДАЛЁННЫЕ
РЕПОЗИТОРИИ**

git clone	Клонирует (копирует) удалённый репозиторий в локальный каталог	git clone https://github.com/пользователь/репозиторий.git
git remote	Управляет подключения ми к удалённым репозитория м.	git remote -v
git fetch	Загружает изменения с удалённого репозитория без слияния с вашей работой.	git fetch origin
git pull	Fetch + Merge. Загружает и сливает изменения из	git pull origin main

СРАВНЕНИЕ

git push	удалённой ветки. Отправляет ваши локальные коммиты в удалённый репозиторий.	git push origin моя-ветка
git diff	Показывает различия между изменениями, коммитами, ветками и т.д.	git diff git diff HEAD~1

Порядок выполнение работы

3.1 Настройка github

Первый шаг — создание учетной записи на GitHub, которая будет использоваться в качестве удаленного сервера для хранения репозиториев.

Инструкция по выполнению:

- 1. Перейдите на сайт <https://github.com/>.
- 2. Нажмите на кнопку "Sign up".
- 3. Следуйте инструкциям на экране: введите вашу электронную почту, создайте пароль и выберите имя пользователя.
- 4. Подтвердите свою учетную запись через письмо, которое придет на указанную почту.



Регистрация на GitHub прошла успешно. Процесс был простым и интуитивно понятным.

настройка git

3.2 Базовая

После установки Git на локальный компьютер вам потребуется выполнить базовую настройку: указать имя пользователя и адрес электронной почты, которые будут отображаться в истории коммитов, а также настроить другие параметры для корректной работы.

```
fernando@ubuntu25-10:~$ git config --global user.name "Alexei FRG"
fernando@ubuntu25-10:~$ git config --global user.email "1132255520@rudn.ru"
fernando@ubuntu25-10:~$ git config --global core.quitepath
fernando@ubuntu25-10:~$ git config --global core.quotepath false
fatal: not in a git directory
fernando@ubuntu25-10:~$ git config --global init.defaultBranch master
fernando@ubuntu25-10:~$ git config --global core.autocrlf input
fernando@ubuntu25-10:~$ git config --global core.autocrlf input
fernando@ubuntu25-10:~$ git config --global core.quotepath false
fernando@ubuntu25-10:~$ git config --global core.safecrlf warn
fernando@ubuntu25-10:~$
```

Базовая настройка Git завершена. Эти команды необходимы для того, чтобы каждый мой коммит (сохранённые изменения) был подписан моим именем и адресом электронной почты. Это очень важно для отслеживания истории изменений, особенно при работе в команде.

3.3 Создание SSH-ключа

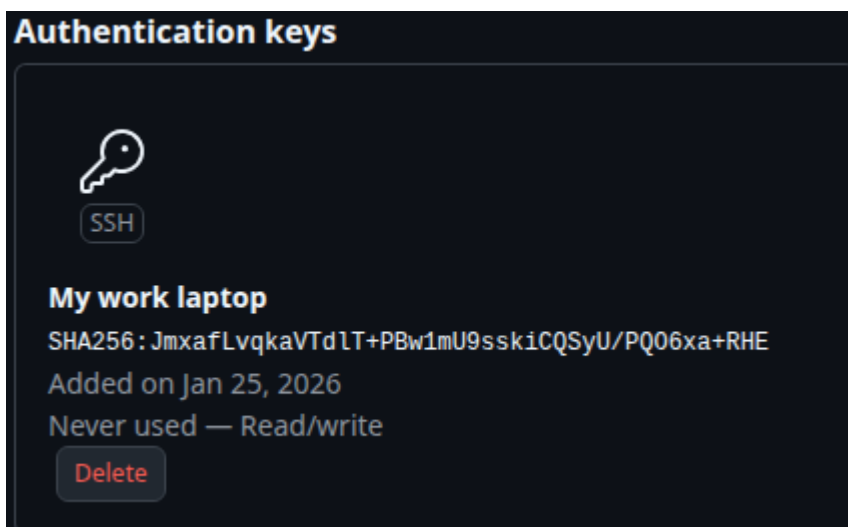
Для безопасного подключения к GitHub без необходимости каждый раз вводить пароль, мы сгенерировали пару SSH-ключей (закрытый и открытый) и добавили открытый ключ в профиль GitHub.

```
fernando@ubuntu25-10:~$ ssh-keygen -C "Alexei FRG 1132255520@rudn.ru"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/fernando/.ssh/id_ed25519):
Enter passphrase for "/home/fernando/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/fernando/.ssh/id_ed25519
Your public key has been saved in /home/fernando/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:JmxafLvqkaVTdLT+PBw1mU9sskiCQSyU/PQ06xa+RHE Alexei FRG 1132255520@rudn.ru
The key's randomart image is:
+--[ED25519 256]--+
|    o.+oo  .  o+ |
|    + + .o. .+= |
|    +..Eo...=. |
|    o  o+. .+... |
|    * S+.  = |
|    + @oo.  . |
|    . =oo. |
|    ++. |
|    .ooo. |
+-----[SHA256]-----+
```

Вот и всё, мы создали ключ, теперь можем скопировать его с помощью этой команды:

```
fernando@ubuntu25-10:~$ cat ~/.ssh/id_ed25519.pub | xclip -sel clip
```

Если нам нужно создать несколько подкаталогов в команде, вы не увидите никаких проблем



Мы сгенерировали SSH-ключ. Это позволяет нам безопасно подключаться к репозиторию GitHub. Я добавил открытый ключ в настройки учетной записи, а закрытый ключ остается на компьютере, обеспечивая безопасное соединение.

3.4 Создание рабочего пространства и репозитория курса

На этом этапе создается репозиторий GitHub на основе предоставленного шаблона, затем формируется локальная структура каталогов для учебных проектов, и удаленный репозиторий клонируется на компьютер.

1. Перейдите на страницу репозитория шаблонов курсов:
2. <https://github.com/yamadharma/course-directory-student-template>
3. Нажмите кнопку «Использовать этот шаблон».
4. В открывшемся окне задайте имя репозитория:
5. study_2025–2026_arch-pc и создайте репозиторий, нажав кнопку «Создать репозиторий из шаблона».

Теперь создана директория, и вы перейдете в нее:

```
fernando@ubuntu25-10:~$ mkdir -p ~/work/study/2025-2026/"Архитектура компьютера"
fernando@ubuntu25-10:~$ cd ~/work/study/2025-2026/"Архитектура компьютера"
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера$
```

Скопируйте созданный на вашем компьютере удалённый репозиторий и скопируйте SSH-ссылку со страницы репозитория.

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера$ git clone --recursive git@github.com:AlexeiFRG1/study_2025-2026_arch-pc.git arch-pc
Cloning into 'arch-pc'...
The authenticity of host 'github.com (140.82.114.3)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TUJJhbpZisF/zLDA0zPMSvHdkr4UvC0QU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 38, done.
remote: Counting objects: 100% (38/38), done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 38 (delta 1), reused 26 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (38/38), 23.57 KiB | 12.00 KiB/s, done.
Resolving deltas: 100% (1/1), done.
Submodule 'template/presentation' (https://github.com/yamadharma/academic-presentation-markdown-template.git) registered for path 'template/presentation'
Submodule 'template/report' (https://github.com/yamadharma/academic-laboratory-report-template.git) registered for path 'template/report'
Cloning into '/home/fernando/work/study/2025-2026/Архитектура компьютера/arch-pc/template/presentation'...
remote: Enumerating objects: 219, done.
remote: Counting objects: 100% (219/219), done.
remote: Compressing objects: 100% (151/151), done.
remote: Total 219 (delta 86), reused 189 (delta 56), pack-reused 0 (from 0)
Receiving objects: 100% (219/219), 2.66 MiB | 378.00 KiB/s, done.
Resolving deltas: 100% (86/86), done.
Cloning into '/home/fernando/work/study/2025-2026/Архитектура компьютера/arch-pc/template/report'...
remote: Enumerating objects: 251, done.
remote: Counting objects: 100% (251/251), done.
remote: Compressing objects: 100% (172/172), done.
remote: Total 251 (delta 111), reused 204 (delta 64), pack-reused 0 (from 0)
Receiving objects: 100% (251/251), 775.12 KiB | 712.00 KiB/s, done.
Resolving deltas: 100% (111/111), done.
Submodule path 'template/presentation': checked out '1c93acf9e731bf186384c85de4aff70037314240'
Submodule path 'template/report': checked out '8ee157c58b3362947b1c71492a65d4dc6882d5ad'
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера$
```

Сначала создайте репозиторий на GitHub, используя веб-интерфейс и

предопределенный шаблон. Затем создайте локальную структуру папок на своем компьютере и клонируйте в нее удаленный репозиторий. Теперь у вас есть локальная копия проекта, связанная с сервером GitHub.

3.5 Настройка каталога курса

Заключительным этапом основной части работы является настройка структуры каталогов курса с помощью команды `make` и отправка исходных файлов на сервер GitHub.

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ echo arch-pc > COURSE
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ make prepare
```

Теперь он перешел в состав проектного совета и завершил подготовку сооружений.

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git add .
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git commit -am 'feat(main): make course structure'
[master 119e320] feat(main): make course structure
248 files changed, 8698 insertions(+), 225 deletions(-)
```

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git push
Enumerating objects: 73, done.
Counting objects: 100% (73/73), done.
Delta compression using up to 4 threads
Compressing objects: 100% (56/56), done.
Writing objects: 100% (70/70), 700.95 KiB | 1.80 MiB/s, done.
Total 70 (delta 24), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (24/24), completed with 1 local object.
To github.com:AlexeiFRG1/study_2025-2026_arch-pc.git
34fed41..119e320 master -> master
```

Вы завершаете настройку структуры курса, а затем используете базовый рабочий процесс Git: добавляете изменения в индекс (`git add`), фиксируете их с содержательным комментарием (`git commit`) и отправляете на удаленный сервер (`git push`). Теперь все локальные изменения синхронизированы с GitHub.

Выполнение самостоятельной работы

Задание 1

1. Создайте отчет по выполнению лабораторной работы в соответствующем каталоге рабочего пространства (`labs/lab02/report`).

```
fernando@ubuntu25-10:~$ cd /home/fernando/work/study/2025-2026/Архитектура\ компьютера/arch-pc/labs/lab01/report
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab01/report$ git status
```

Задание 2

2. Скопируйте отчеты по выполнению предыдущих лабораторных работ в соответствующие каталоги созданного рабочего пространства.

```
fernando@ubuntu25-10:~$ cp ~/Downloads/lab01_report.pdf "/home/fernando/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab01/report/"
```

Задание 3

3. Загрузите файлы на github.

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab01/report$ git add .
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab01/report$ git commit
-m "feat(labs): add report for lab02 and copy lab01"
[master 0a5db4f] feat(labs): add report for lab02 and copy lab01
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 labs/lab01/report/lab01_report.pdf
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab01/report$ git pudh
git: 'pudh' is not a git command. See 'git --help'.

The most similar command is
push
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab01/report$ git push
Enumerating objects: 10, done.
```

Выводы

В этой лабораторной работе я изучил теорию и практику системы Git. Я научился настраивать Git, создавать репозитории на GitHub, использовать SSH-ключи для безопасности, а также делать основные команды: add, commit и push. Все цели работы были выполнены..

Список литературы

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Disponible en: <https://git-scm.com/book/en/v2>

Pilón, M. A. (2017). Control de versiones con Git y GitHub. Universidad Autónoma de Madrid. Servicio de Publicaciones.

Loeliger, J., & McCullough, M. (2012). Version Control with Git (2nd ed.). O'Reilly Media.

Spinellis, D. (2012). Code Quality: The Open Source Perspective. Addison-Wesley. (Capítulo sobre gestión de configuración y control de versiones).

Cervantes, H. (2015). Sistemas de Control de Versiones: Git. En Desarrollo Colaborativo de Software (pp. 45-80). Universidad Nacional Autónoma de México.

Torvalds, L., & Hamano, J. (2010). The Git source code management system. Documentación oficial y código fuente. Disponible en: <https://git-scm.com/>

Driessen, V. (2010). A successful Git branching model. [Blog post]. Conocido popularmente como "Git Flow". Disponible en: <https://nvie.com/posts/a-successful-git-branching-model/>

GitHub. (2023). GitHub Docs: Using Git. Documentación práctica sobre el uso de Git en GitHub. Disponible en: <https://docs.github.com/es/get-started/using-git>